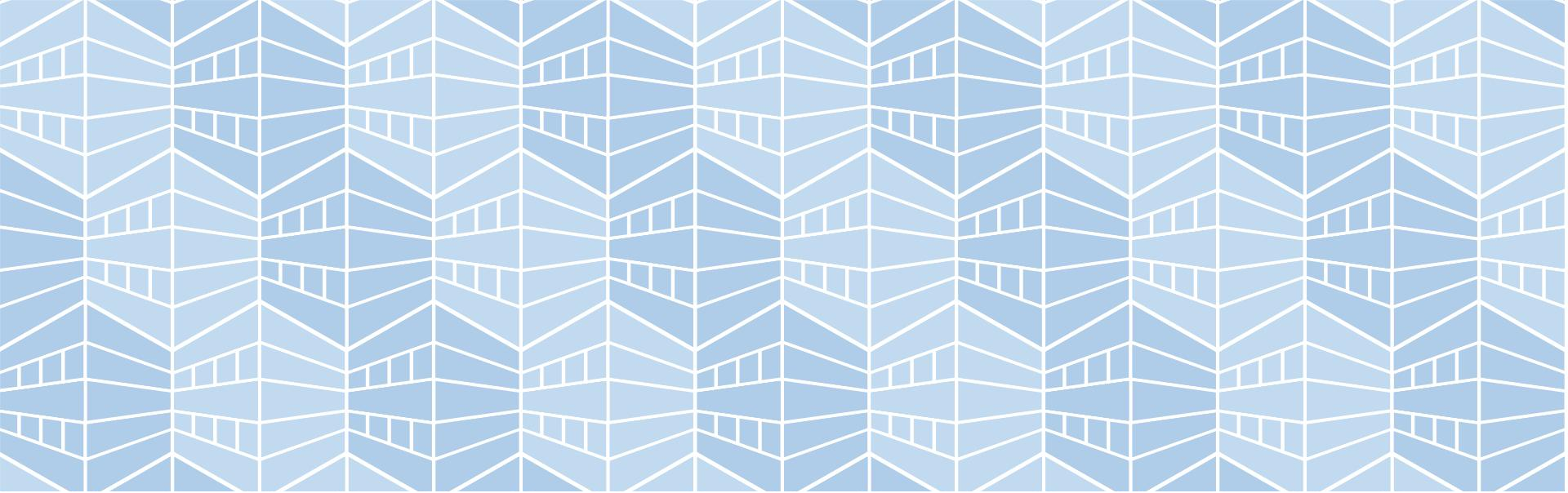




Functional Programming in Serverless World

Lambda Days 2018 - Cracow (23.02.2018)

~ # whoami

(afronski)

- ✓ Software and Operations Engineer
[Appliscale](#)

Erlang, Elixir, Node.js
AWS (2 certificates)

- ✓ **Functional Miners** co-organizer



*P.S. **Appliscale** is hiring!*

What is
serverless?

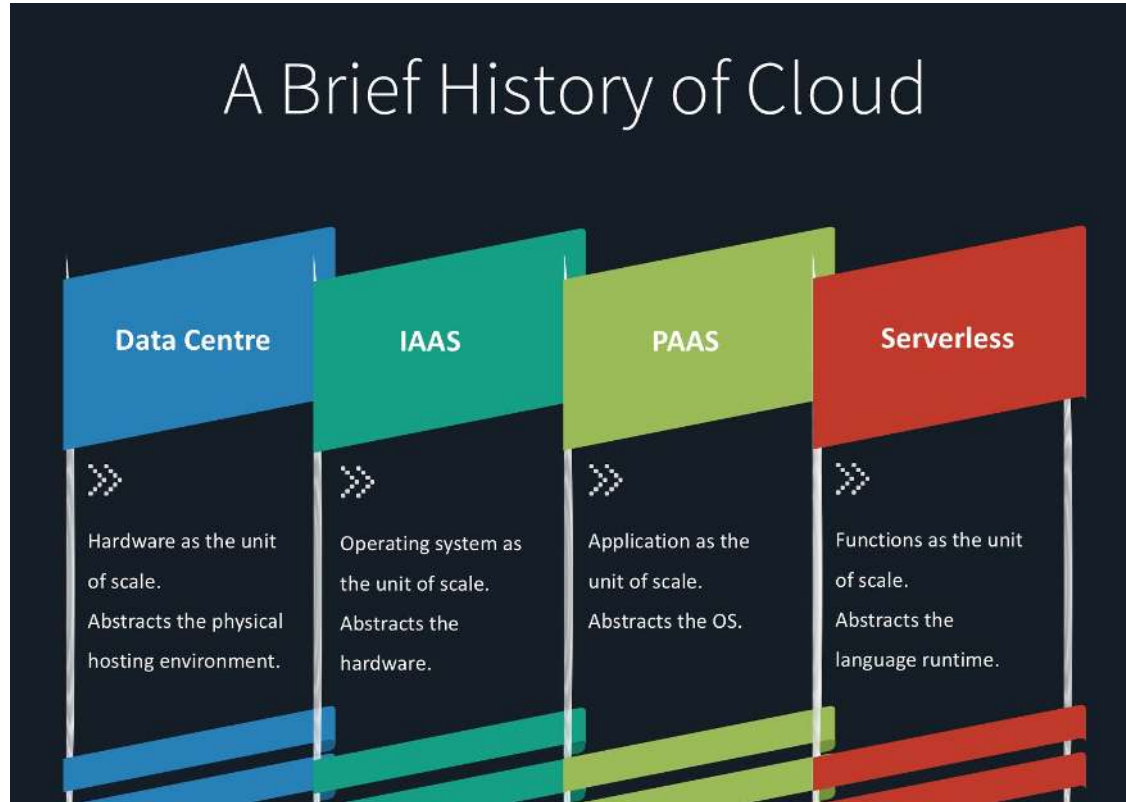


<http://celiacandthebeast.com/wp-content/uploads/2014/08/SRSLY.jpg>

There are
servers

There are
ops

A Brief History of Cloud

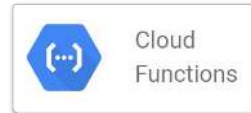


read.acloud.guru/iaas-paas-serverless-the-next-big-deal-in-cloud-computing

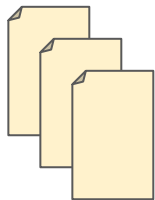
Serverless

Back-end as a Service
(**BaaS**)

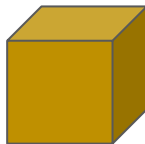
Function as a Service
(**FaaS**)



Source Code



Package



Configuration

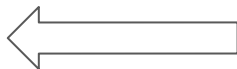


RAM: 512 MB

Function (*Execution Unit*)



Response



RAM: 512 MB
0.5 GB

Real time: 120 ms
Bill time: 200 ms (buckets by 100 ms)

Price: 0.00001667 GB * seconds

**Cost: 0.00001667 * 0.5 GB * 0.2 s =
0.000001667 USD =
1.7 μUSD**



Incoming Event

Why
serverless?

Serverless Computing: Economic and Architectural Impact

Gojko Adzic
Neuri Consulting LLP
25 Southampton Buildings
London, United Kingdom WC2A 1AL
gojko@neuri.co.uk

Robert Chatley
Imperial College London
180 Queen's Gate
London, United Kingdom SW7 2AZ
rbc@imperial.ac.uk

<http://www.doc.ic.ac.uk/~rbc/papers/fse-serverless-17.pdf>

[*Gojko Adzic - Designing for the Serverless Age \(GOTO 2017\)*](#)

Costs

Service instance	Billable unit	Unit cost (USD)	Fail-over costs (%)	Cost of 12 x 200ms exec'ns	% reference price
Lambda (128 MB)	100 ms	\$0.000000208	included	\$0.000004992	24.94%
Lambda (512 MB)	100 ms	\$0.000000834	included	\$0.000020016	100.00%
Heroku Hobby (512 MB)	1 month	\$7.00	100%	\$0.0097222222	48572.25%
AWS EC2 t2.nano (512 MB)	1 hour	\$0.0059	100%	\$0.0118	58952.84%
AppEngine B1 (128MB)	1 hour	\$0.05	100%	\$0.1	499600.32%
AppEngine B4 (512MB)	1 hour	\$0.20	100%	\$0.4	1998401.28%

Architecture

[...] breaking the monolith into functions that could be independently deployed, meant that they were **better able to** split the team up to work on more things in **parallel**, and to deploy each feature **separately**.

Operations

[...] so their estimate is that moving to Lambda gave an **operational cost reduction** of **greater than 95%** for a comparable amount of compute.




A CLOUD GURU

[Courses](#)
[Discussions](#)
[Teams](#)
[Serverless](#)
[Blog](#)



afonaki

Act On Your New Years Resolutions

Make 2017 the year to focus on your career and get started now!

[Your Courses](#)
[All Courses](#)

AWS ASSOCIATE CERTIFICATIONS

[Are you a business? Check out our training for teams.](#)









Certified SysOps Administrator - Associate 2017

Learn to manage infrastructure on Amazon Web Services and pass the SysOps Associate exam!

0% complete [Continue](#)



Certified Developer - Associate 2017

Learn how to develop/script on Amazon Web Services and pass the DVA Associate exam!

0% complete [Continue](#)





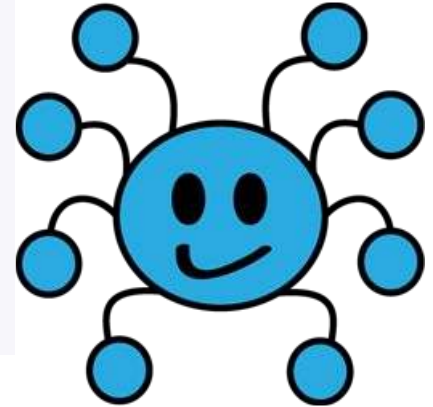
droplr

Droplr Serverless Revolution

How we killed 50 servers in a year

Antoni Orfin <antoni@droplr.com>
Solutions Architect @ Droplr

 **serverless**



NORDSTROM

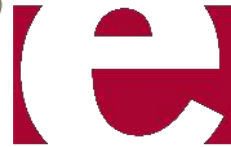


Technology

 **Scala** 



 elixir


ERLANG



 PURESCRIPT





Google Cloud Platform



Shims

```
const spawn = require("child_process").spawn;

exports.handler = function(event, context) {
  process.env["PATH"] = process.env["PATH"] + ":" + process.env["LAMBDA_TASK_ROOT"]
  process.env["LD_LIBRARY_PATH"] = process.env["LAMBDA_TASK_ROOT"]
  const main = spawn("./exec", { stdio: ["pipe", "pipe", process.stderr] });

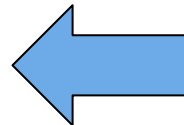
  main.stdout.on("data", function(data) {
    context.done(null, data.toString());
  });

  main.on("close", function(code) {
    context.done(null, code);
  });

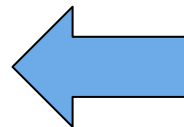
  main.on("exit", function(code){
    context.done(null, code);
  });

  main.on("error", function(err) {
    context.done(null, err);
  });

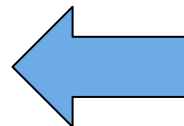
  main.stdin.write(JSON.stringify({ event, context }) + "\n");
}
```



Using Node.js *process* API to spawn an executable



Handling events reported by subprocess and passing results to the AWS *Lambda* handler



Sending *input arguments* to the spawned subprocess

 **Scala** 



 **APPLISCALE**



 elixir


ERLANG



 **PURESRIPT**



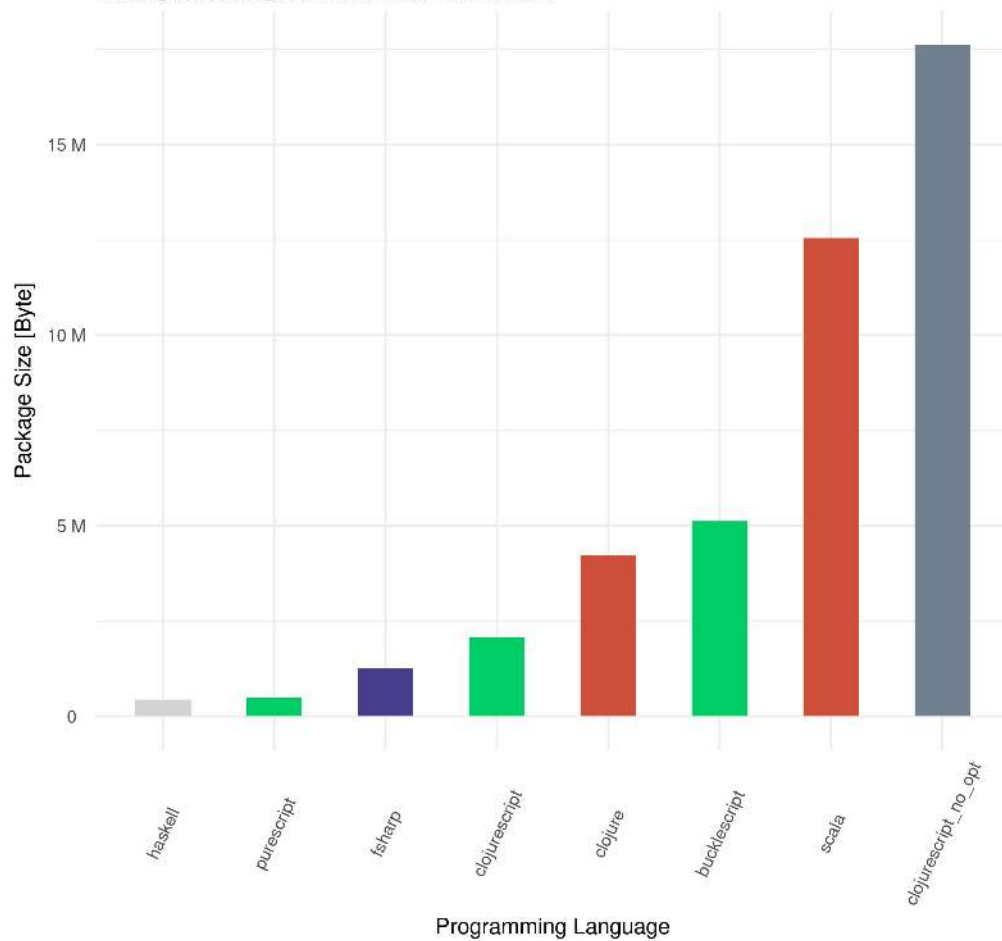
Measurements

Test Scenarios

1. **Two endpoints:**
 - a. Echoing incoming body.
 - b. Sieve of Erathos (naive implementation).
2. **Package Size.**
3. **Memory Usage** (dependent on input argument).
4. **Execution Time** (dependent on input argument).
5. **Startup Time** (cold start, no work done).

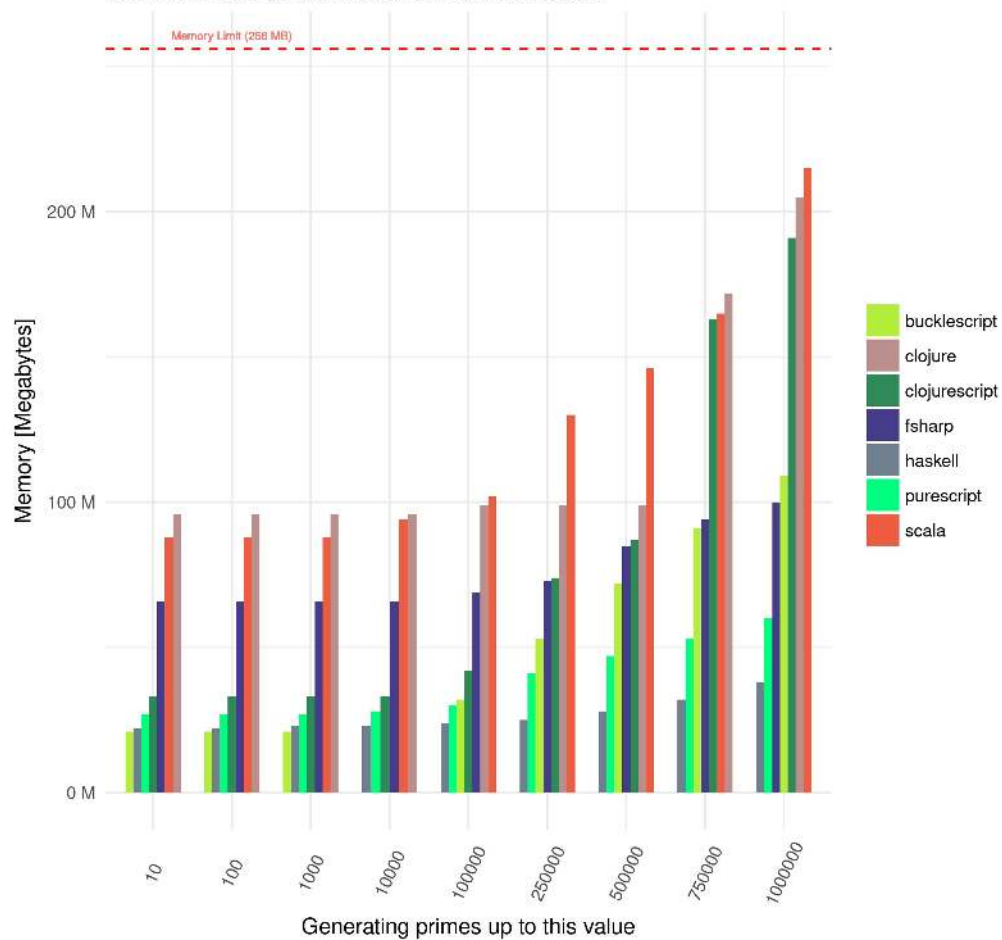
Package Size

How big is a package uploaded to the FaaS service?



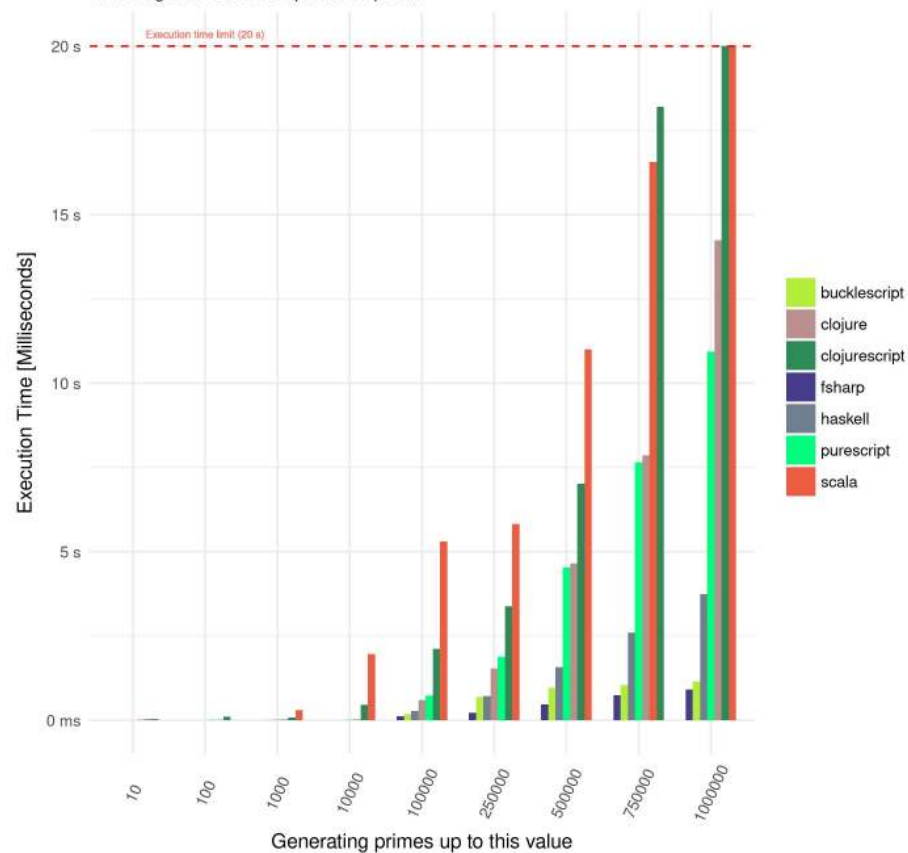
Memory Usage

How much memory was allocated for a specific request?



Execution Time

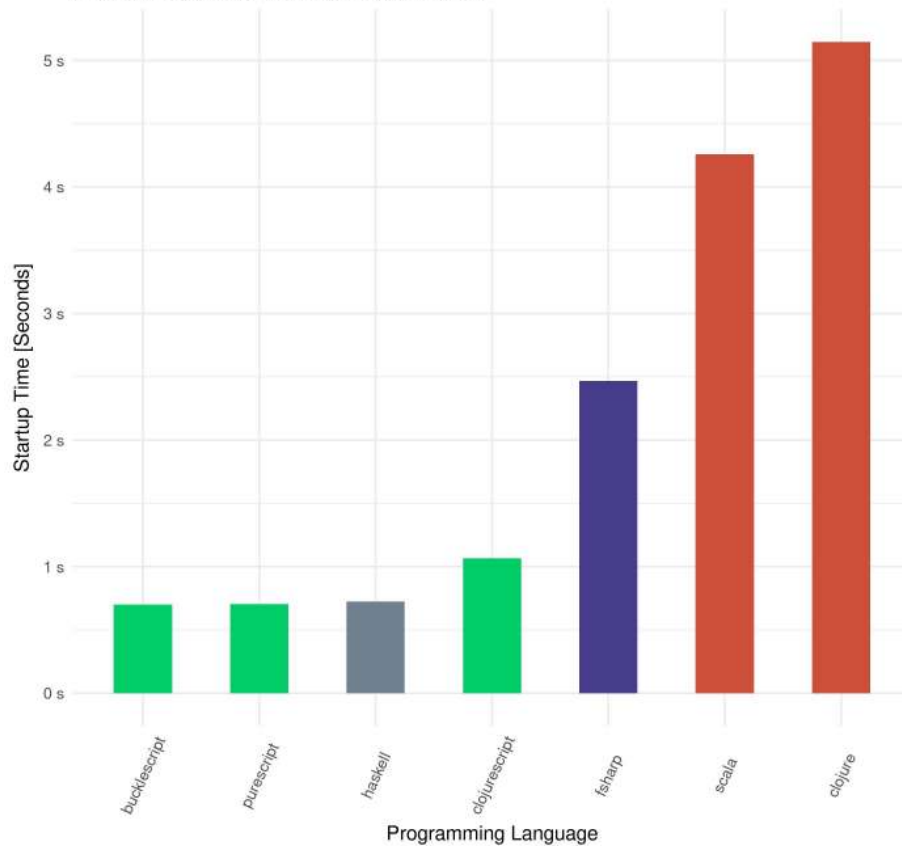
How long code ran for a specific request?



"Situated" Programs (from Rich Hickey's keynote at Clojure/Conj 2017)

Startup Time

How slow is the first call of our function (cold start)?



[How long does AWS Lambda keep your idle functions around before a cold start?](#)

Constraints

Constraints

1. **No knowledge** about container and its reuse.
2. New approach requires **new paths**.
3. Limits **everywhere!**
4. Provider **limitations**.
5. **VM optimizations** required from the get go.
6. Beware the **toolchain!**

Let's recap!

Safe default:

F# on Azure or AWS

(.NET Core 2.0)

Serverless Computing: Economic and Architectural Impact

Gojko Adzic
Neuri Consulting LLP
25 Southampton Buildings
London, United Kingdom WC2A 1AL
gojko@neuri.co.uk

Robert Chatley
Imperial College London
180 Queen's Gate
London, United Kingdom SW7 2AZ
rbc@imperial.ac.uk

<http://www.doc.ic.ac.uk/~rbc/papers/fse-serverless-17.pdf>

[*Gojko Adzic - Designing for the Serverless Age \(GOTO 2017\)*](#)

Context is **King**



Thank you!

Questions?

1. Our company - [Appliscale](#) and [job offers](#).
2. Me - [afronski.pl](#) and my [talks](#).
3. **Functional Miners** meetup ([facebook](#), [twitter](#), [github](#), [email](#)).
4. [Serverless Architecture](#), [Serverless](#) (Martin Fowler's articles).
5. [AWS Lambda](#), [Google Cloud Functions](#), [Azure Functions](#).
6. [Container Reuse in AWS Lambda](#).
7. [Running Executables in AWS Lambda](#).
8. [Serverless Computing: Economic and Architectural Impact](#).
9. [Why the JVM is a Good Choice for Serverless Computing](#).
10. [Droplr Serverless Revolution](#).
11. [Optimizing Enterprise Economics with Serverless](#).
12. [Should I use AWS Lambda or EC2?](#)
13. [AWS Lambda support for .NET Core 2.0](#).
14. [Repository](#) with examples, scripts, and measurements.

References